

仿真平台使用说明

进入 U 盘系统

1. 进入 BIOS

插入系统 U 盘，然后重启电脑。在开机过程中，根据屏幕提示按下相应的按键（通常是 Del、F2、Esc 或 F10 等）进入 BIOS 设置界面。不同品牌和型号的主板进入 BIOS 的按键可能不同，常见的按键包括：

Del：大多数主板使用此键。

F2：部分华硕（ASUS）和技嘉（Gigabyte）主板使用此键。

Esc：部分惠普（HP）和联想（Lenovo）笔记本使用此键。

F10：部分戴尔（Dell）和惠普（HP）笔记本使用此键。

2. 设置启动项

找到启动设置：进入 BIOS 后，找到“启动顺序”（Boot Order）或“启动设备”（Boot Device）选项。设置 U 盘为第一启动项：

在启动顺序列表中，找到 U 盘设备（通常显示为“USB Drive”或类似名称）。将 U 盘设置为第一启动项。通常可以通过上下箭头键选择设备，然后按“+”或“-”键调整顺序，或者直接选择“设置为第一启动项”选项。

PhoenixBIOS Setup Utility							
Main	Advanced	Security	Boot	Exit			
Removable Devices +Hard Drive CD-ROM Drive Network boot from Intel E1000				Item Specific Help			
				Keys used to view or configure devices: <Enter> expands or collapses devices with a + or - <Ctrl+Enter> expands all <+> and <-> moves the device up or down. <n> May move removable device between Hard Disk or Removable Disk <d> Remove a device that is not installed.			
F1	Help	↑↓	Select Item	-/+	Change Values	F9	Setup Defaults
Esc	Exit	↔	Select Menu	Enter	Select ► Sub-Menu	F10	Save and Exit

3. 启用 USB 启动功能

检查 USB 启动功能：在 BIOS 设置中，找到“安全启动”（Secure Boot）或“启动选项”（Boot Options）。

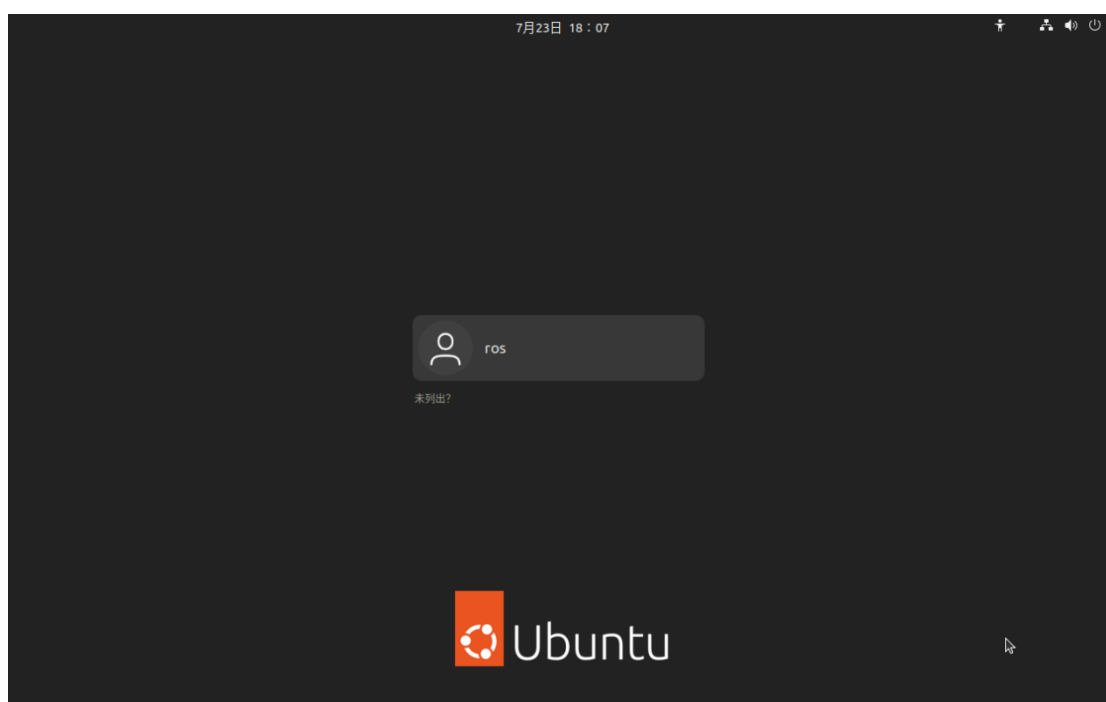
如果“安全启动”已启用，建议先禁用它，因为某些 U 盘启动盘可能不支持安全启动。

确保“USB 启动”或“从 USB 设备启动”选项已启用。如果未启用，即使 U 盘插入，也可能无法从 U 盘启动。

4. 保存设置并重启

保存设置：完成上述设置后，按“F10”键（或 BIOS 界面中提示的保存键）保存设置并退出 BIOS。

重启电脑：电脑将自动重启，并尝试从 U 盘启动。如果 U 盘启动盘制作正确，系统将从 U 盘加载启动界面。因 U 盘读写速度限制，加载过程大概黑屏 2~3 分钟。



系统密码为：ros

文件系统扩容(首次进入系统必须执行)

烧录完成后系统并没有完全使用硬件容量，需要对文件系统扩容。



使用命令 `df -h` 查看根目录文件系统大小，如图显示 117G 表示已经扩容，如果小于需要按照下面步骤对文件系统扩容。

```
ros@ubuntu:~$ df -h
```

文件系统	大小	已用	可用	已用%	挂载点
tmpfs	387M	12M	376M	4%	/run
/dev/sda2	117G	14G	99G	12%	/
tmpfs	1.9G	0	1.9G	0%	/dev/shm
tmpfs	5.0M	4.0K	5.0M	1%	/run/lock
efivarfs	256K	57K	195K	23%	/sys/firmware/efi/efivars

执行根目录 `expand_rootfs_up.sh` 脚本对文件系统扩容

扩容完成后输出如下：

```
ros@ubuntu:~$ ls *.sh
expand_rootfs_up.sh
ros@ubuntu:~$ ./expand_rootfs_up.sh
Please run as root
ros@ubuntu:~$ sudo ./expand_rootfs_up.sh
Detected root device: /dev/sdb2
Using device: /dev/sdb, partition: 2
NOCHANGE: partition 2 is size 249545367. it cannot be grown
ros@ubuntu:~$
```

如果没有执行脚本对文件系统扩容后续操作会导致系统空间不足，无法写入，甚至不能开机。

补救措施

如果已经出现异常可以参考下面的方法

可以进入系统

```
File "/usr/bin/colcon", line 33, in <module>
    sys.exit(load_entry_point('colcon-core==0.12.1', 'console_scripts', 'colcon'))
File "/usr/lib/python3/dist-packages/colcon_core/command.py", line 118, in main
    return _main(command_name=command_name, argv=argv)
File "/usr/lib/python3/dist-packages/colcon_core/command.py", line 185, in _main
    create_log_path(args.verb_name)
File "/usr/lib/python3/dist-packages/colcon_core/location.py", line 186, in create_log_path
    os.makedirs(str(path))
File "/usr/lib/python3.10/os.py", line 225, in makedirs
    mkdir(name, mode)
OSError: [Errno 28] No space left on device: 'log/build_20251009-09_04-01-40'
```

如果可以进入系统，写入操作都会报错，如 `apt install`, `pip install`, 创建修改文件等，

都会出现类似空间不足的提示如，**No space left on device**。此时甚至不能运行扩容脚本。

解决办法：删除一些文件，或者卸载一些软件，扩容之后重新安装

一些可删除的文件

```
sudo rm -rf /var/log/*
sudo rm -rf ~/.vscode_server ~/.ros ~/.cache
```

apt 卸载一些软件

```
sudo apt remove ros-humble-rqt*
```

之后按提示重新运行 `expand_rootfs_up.sh`

```
cd ~/
sudo ./expand_rootfs_up.sh
```

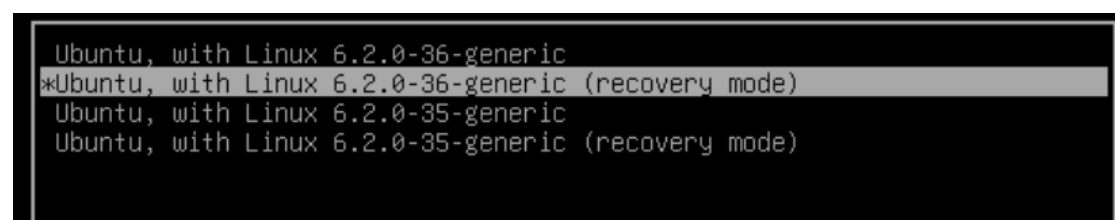
不能进入系统

表现为 Ubuntu 开机卡在 `/dev/sda* : clean , ***files , ***blocks`，需要找到 `/dev/sda*` 这个分区删掉一些没用的文件

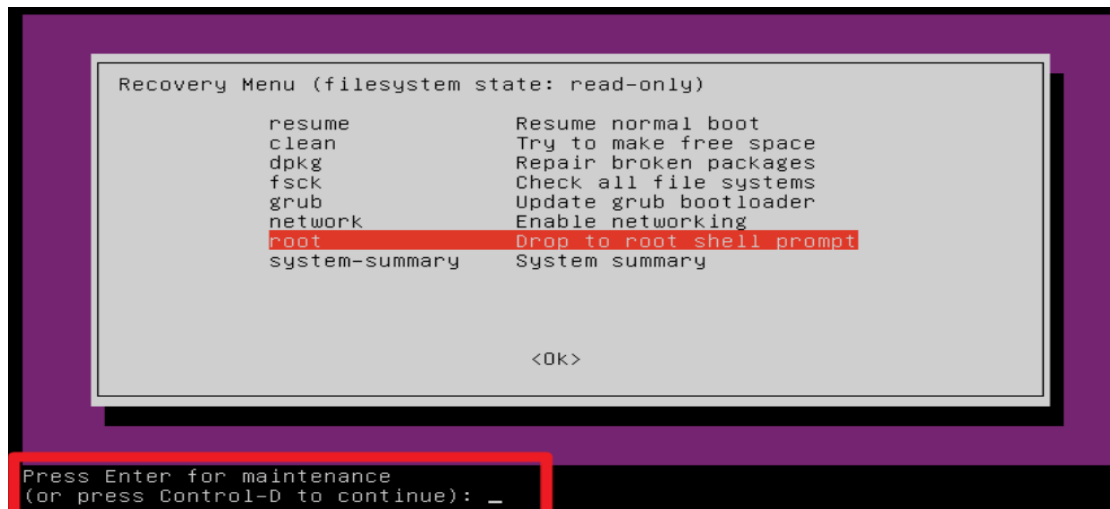
开机再 GRUB 进入高级选项



选择恢复模式



选择 root 回车，参考什么步骤删除一些文件，或者卸载一些软件，扩容之后重新安装



之后按提示重新运行 `expand_rootfs_up.sh`

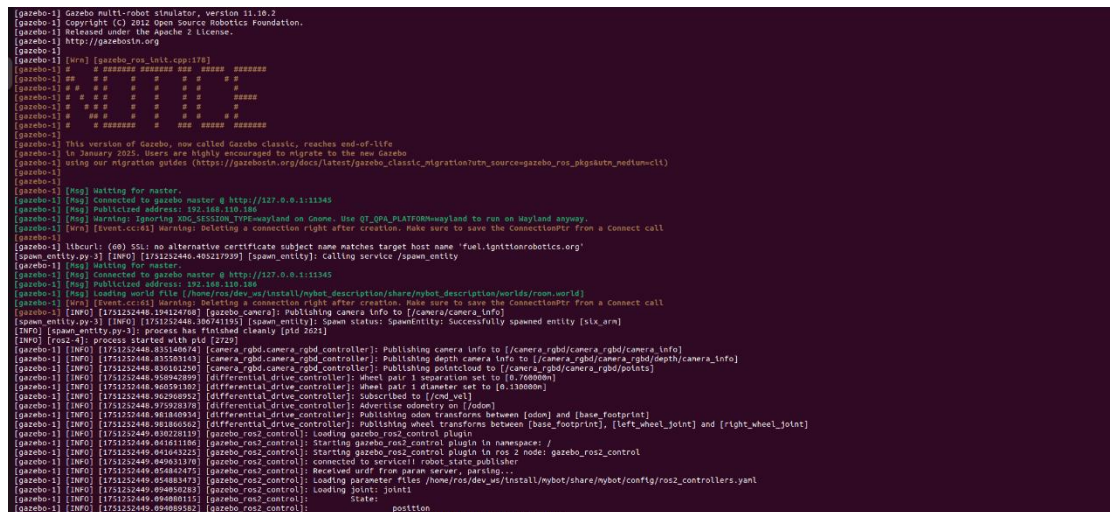
```
cd /home/ros
./expand_rootfs_up.sh
```

基础案例

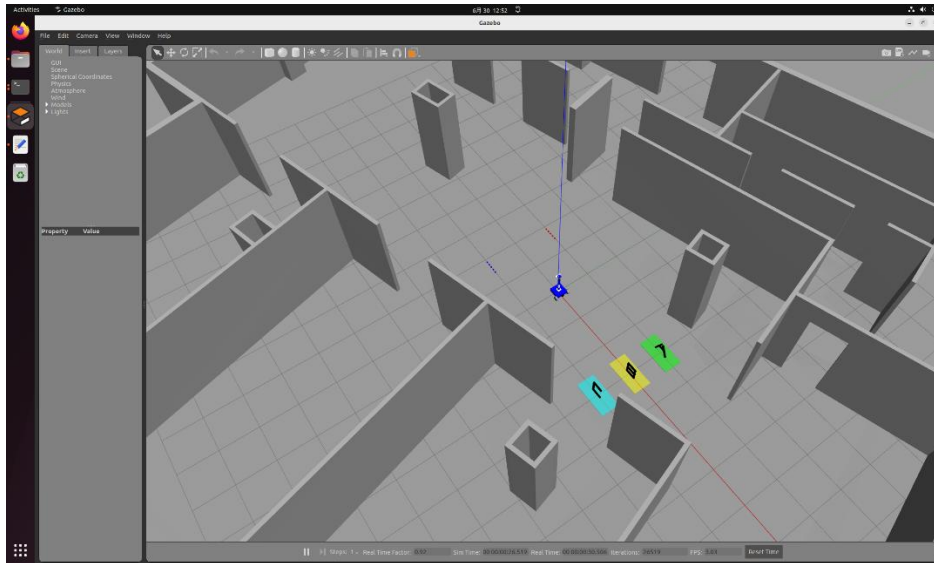
包含基础的移动底盘加六轴机械臂的仿真，雷达、相机、rgbd 相机传感器仿真；功能包括 MoveIt2 机械臂控制、夹爪抓取、以及 Nav2 导航等功能。

1.启动仿真(机械臂+底盘+传感器)

```
ros2 launch mybot gazebo_world.launch.py
```



启动后会显示



新建终端，输入 `ros2 topic list`

可以看到很多话题名称，其中

```
/arm_controller/state # 获取机械臂实时状态
/arm_controller/joint_trajectory # 发送机械臂关节轨迹指令。
/camera/image_raw #相机数据话题
/camera_rgbd/camera_rgbd/depth/image_raw #rgbd 深度图数据
/camera_rgbd/camera_rgbd/image_raw #rgbd 图像数据
/camera_rgbd/camera_rgbd/points #rgbd 点云数据
/cmd_vel #速度控制话题
/odom # 里程计数据话题
/scan # 雷达数据话题
```

输入 `ros2 service list` 查看服务其中

```
/ATTACHLINK
/DETACHLINK
```

这两个控制夹爪夹取方块是绑定方块到夹爪上。

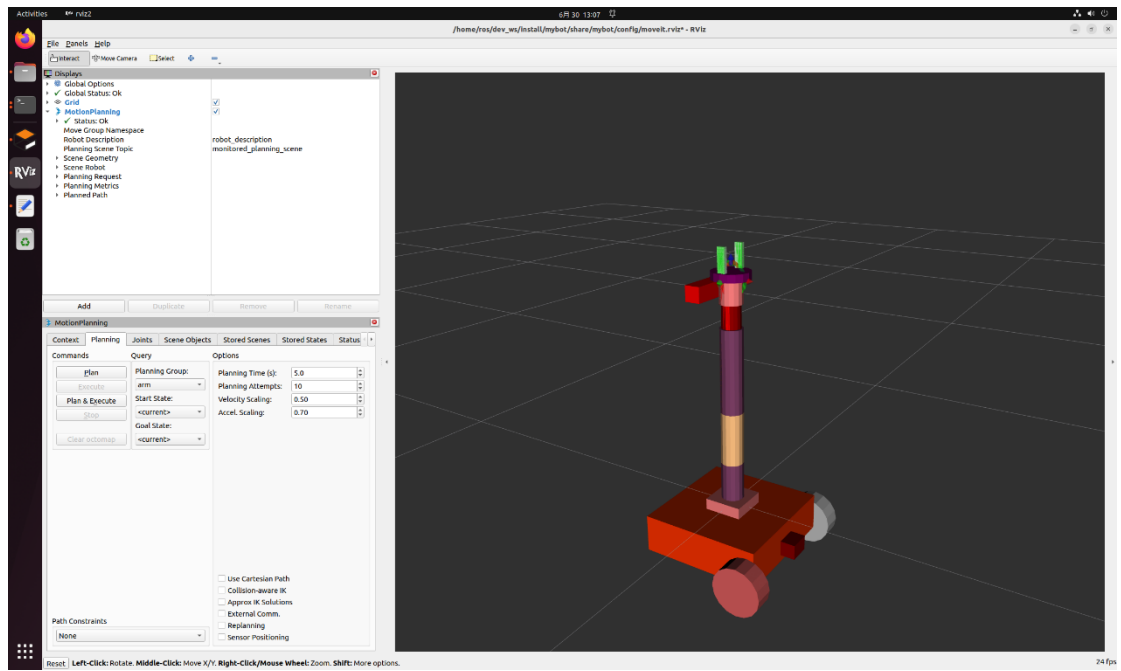
输入 `ros2 action list`

```
/arm_controller/follow_joint_trajectory
/gripper_controller/follow_joint_trajectory
```

这两个可以用于控制机械臂关节状态和夹爪关节状态

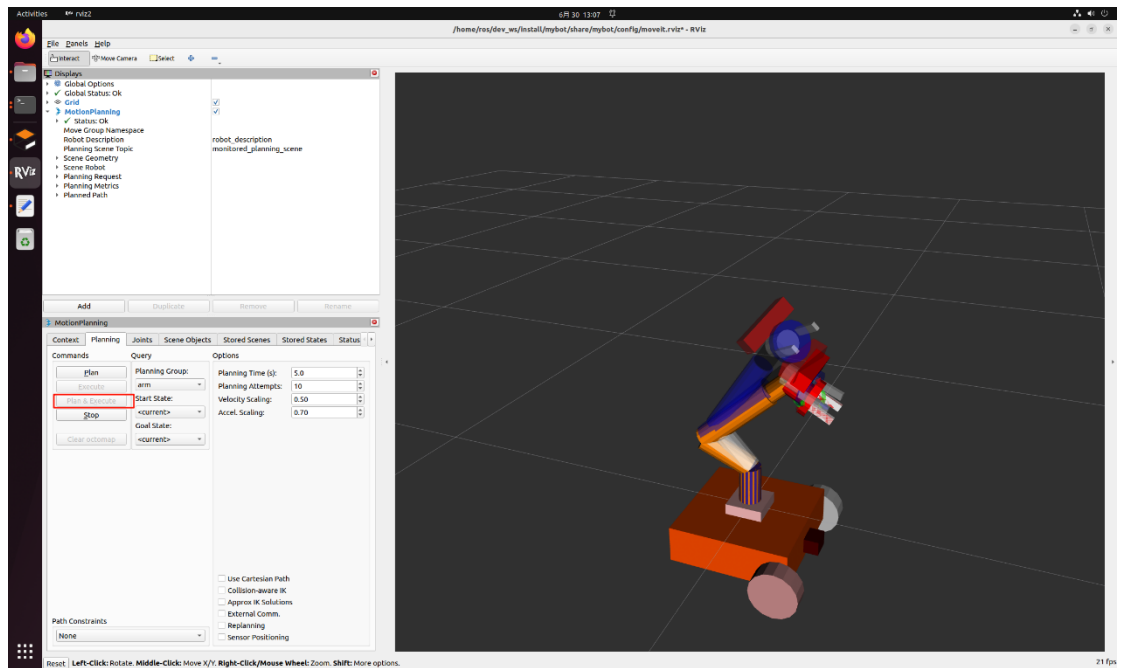
2.启动 moveit(机械臂规划、控制)

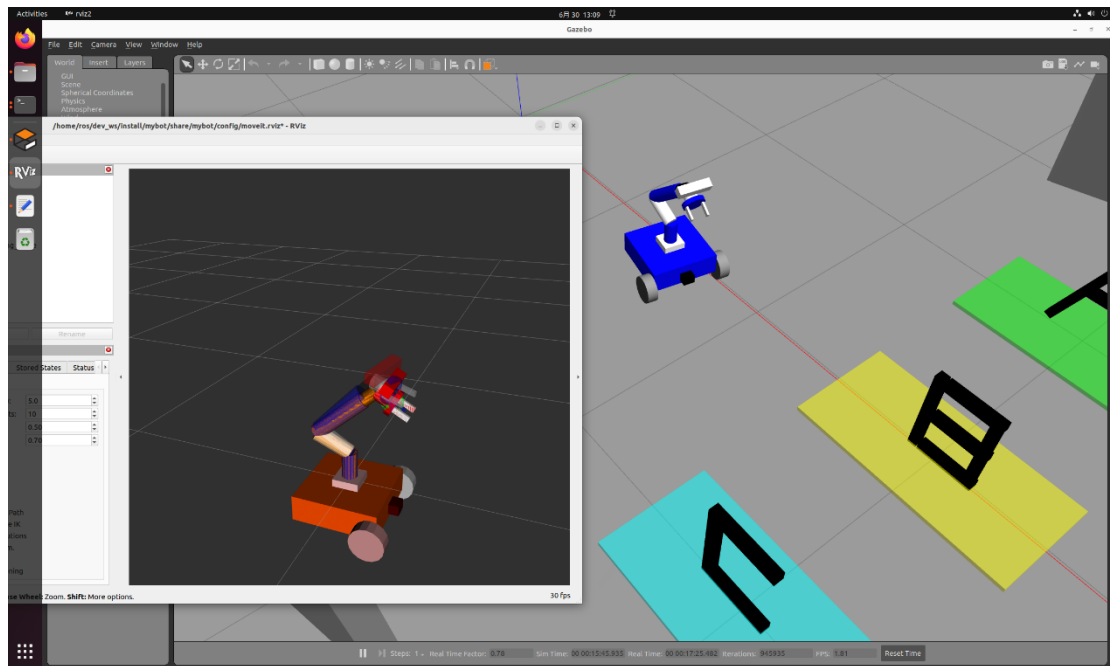
```
ros2 launch mybot my_moveit_rviz.launch.py
```



可以看到机械臂当前的状态，和 moveit 控制面板

调整机械臂末端位置，点击面板 Plan & Execute 执行动作，机械臂会运动到目标位置。





核心 MoveIt! 服务（重点关注）

1. 运动规划服务:

- `/compute_ik`
类型: `moveit_msgs/srv/GetPositionIK`
作用: 计算逆运动学（给定末端位姿求关节角度）
- `/compute_fk`
类型: `moveit_msgs/srv/GetPositionFK`
作用: 计算正运动学（给定关节角度求末端位姿）
- `/compute_cartesian_path`
类型: `moveit_msgs/srv/GetCartesianPath`
作用: 计算笛卡尔路径（直线运动规划）

核心 MoveIt! 动作

动作接口比服务更适用于长时间运行的任务:

1. `/move_action`

类型: `moveit_msgs/action/MoveGroup`

作用: 主运动规划接口，发送目标位姿后自动完成:

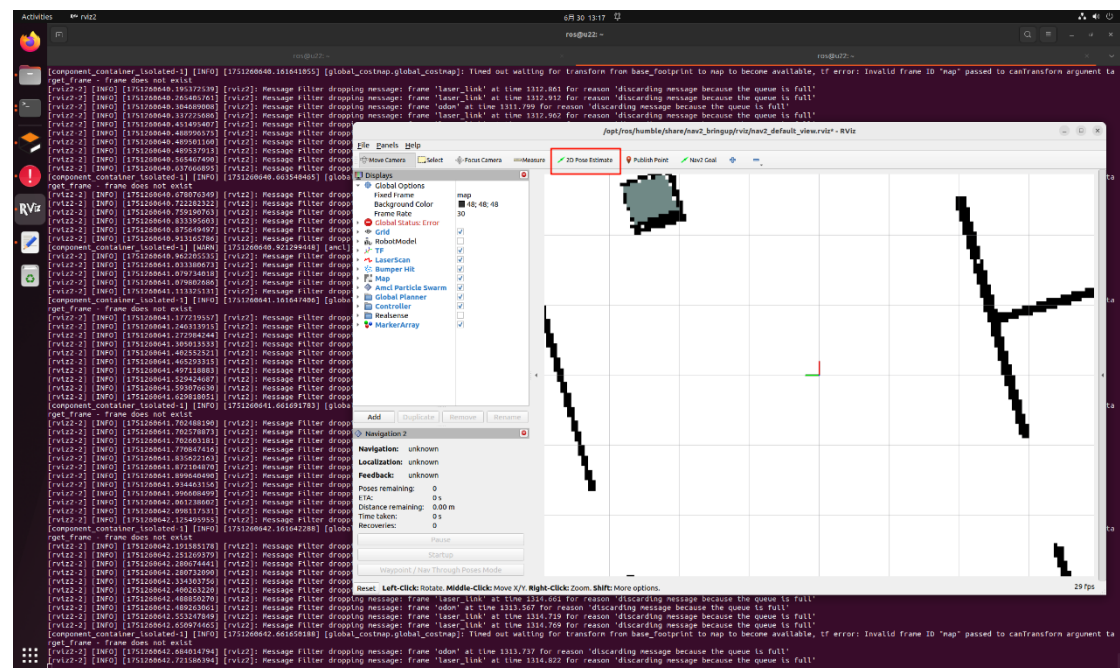
- 路径规划
- 轨迹优化
- 碰撞检测

- 执行控制

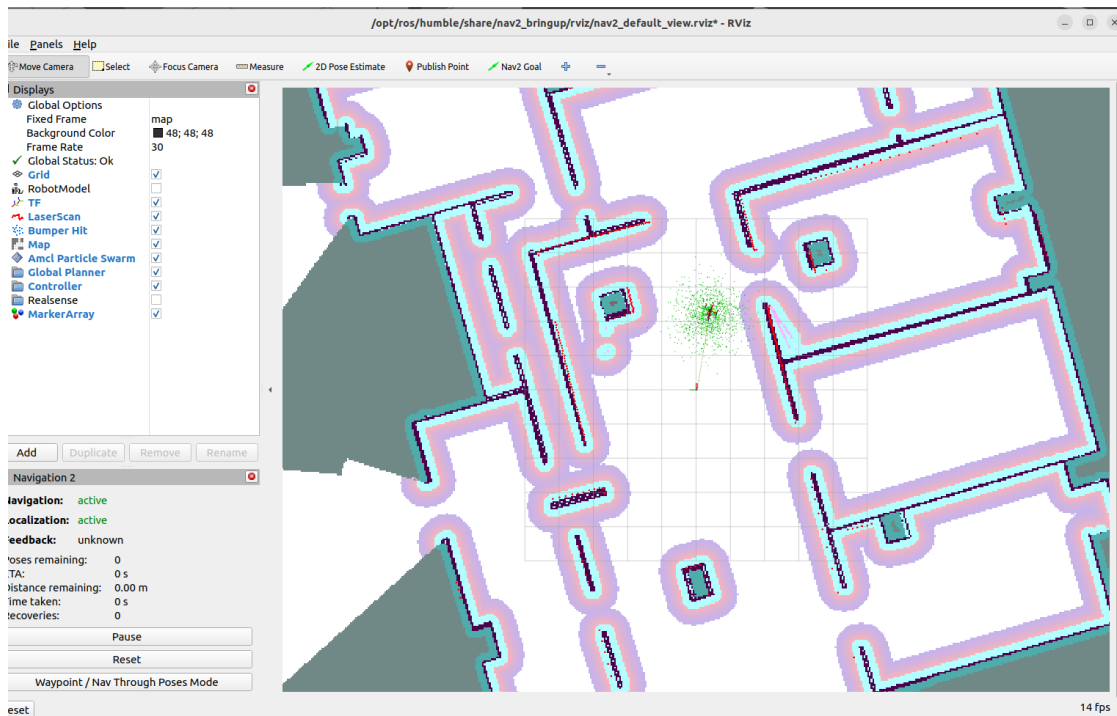
3.启动 nav2(导航)

```
ros2 launch bot_navigation nav_bringup_gazebo.launch.py
```

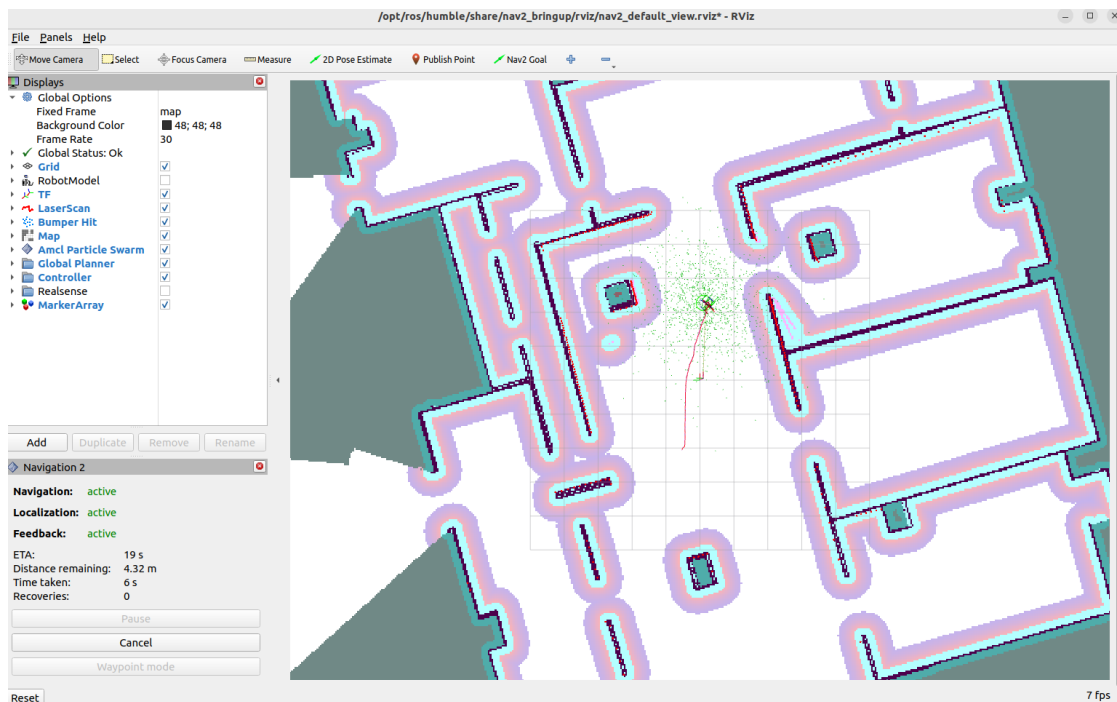
启动后终端已知滚动日志可以忽略，使用 2D Pose Estimate 工具设置初始位置。



可以根据雷达数据多次调整，直到雷达扫描的轮廓和地图大致重合



使用 Nav2 Goal 工具在地图上拖动，发布目标点和机器人朝向，之后地图上会显示运动轨迹，机器人会避开雷达扫描到的障碍物，到达目标点



核心导航动作 (Actions)

这些是 Nav2 的主要导航接口：

1. **/navigate_to_pose**
类型: **nav2_msgs/action/NavigateToPose**
作用: 主导航接口, 导航到指定位置 (最常用)
 2. **/navigate_through_poses**
类型: **nav2_msgs/action/NavigateThroughPoses**
作用: 依次导航通过多个航点
 3. **/follow_waypoints**
类型: **nav_msgs/action/FollowWaypoints**
作用: 导航通过一系列航点 (类似 **navigate_through_poses**)
 4. 底层控制动作:
 - **/backup** - 向后移动指定距离
 - **/spin** - 原地旋转指定角度
 - **/drive_on_heading** - 沿指定方向移动
 - **/wait** - 等待指定时间
 5. 路径规划动作:
 - **/compute_path_to_pose** - 计算到目标位置的路径
 - **/compute_path_through_poses** - 计算通过多个点的路径
 - **/smooth_path** - 路径平滑优化
-

核心导航服务 (Services)

1. 地图服务:
 - **/map_server/load_map**
类型: **nav2_msgs/srv/LoadMap**
作用: 动态加载新地图
 - **/save_map**
类型: **nav2_msgs/srv/SaveMap**
作用: 保存当前地图
2. 定位服务:

- `/reinitialize_global_localization`
类型: `std_srvs/srv/Empty`
作用: 重设定位 (AMCL 粒子群)
- `/set_initial_pose`
类型: `geometry_msgs/msg/PoseWithCovarianceStamped`
作用: 设置初始定位位置

3. 代价地图服务:

- `/global_costmap/clear_entirely_global_costmap`
作用: 清除全局代价地图
- `/local_costmap/clear_entirely_local_costmap`
作用: 清除局部代价地图

4. 生命周期管理:

- `/lifecycle_manager_navigation/manage_nodes`
类型: `lifecycle_manager_msgs/srv/ManageLifecycleNodes`
作用: 启动/停止导航节点

关键话题 (Topics)

1. 定位相关:

- `/amcl_pose` - AMCL 定位结果
- `/particle_cloud` - 定位粒子云 (可视化用)
- `/initialpose` - 设置初始位姿的话题

2. 路径规划:

- `/plan` - 全局规划路径
- `/plan_smoothed` - 平滑后的路径
- `/global_plan` - 全局路径 (Nav2 内部)

3. 控制与执行:

- `/cmd_vel_nav` - 导航系统生成的最终速度命令
- `/cmd_vel_teleop` - 手动遥操作的速度命令
- `/local_plan` - 局部规划结果

4. 地图与成本图:

- `/map` - 当前使用的地图
- `/global_costmap/costmap` - 全局成本图
- `/local_costmap/costmap` - 局部成本图

5. 目标点:

- `/goal_pose` - 导航目标位置
- `/waypoints` - 航点序列

4.大模型对话

测试

```
llama-cli -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -n 512 -c 2048 -sys "你是编程小助手" -co -cnv --threads 8
```

```
system_info: n_threads = 8 (n_threads_batch = 8) / 2 | CPU : SSE3 = 1 | SSSE3 = 1 | AVX = 1 | AVX_VNNI = 1 | AVX2 = 1 | F16C = 1 | FMA = 1 | BMI2 = 1 | LLAMAFILE = 1 | OPENMP = 1 | REPACK = 1 |
main: interactive mode on.
sampler seed: 1379855646
sampler params:
  repeat_last_n = 64, repeat_penalty = 1.000, frequency_penalty = 0.000, presence_penalty = 0.000
  dry_multiplier = 0.000, dry_base = 1.750, dry_allowed_length = 2, dry_penalty_last_n = 2048
  top_k = 40, top_p = 0.950, min_p = 0.050, xtc_probability = 0.000, xtc_threshold = 0.100, typical_p = 1.000, top_n_sigma = -1.000, temp = 0.800
  mirostat = 0, mirostat_lr = 0.100, mirostat_ent = 5.000
sampler chain: logits -> logits-bias -> penalties -> dry -> top-n-sigma -> top-k -> typical -> top-p -> min-p -> xtc -> temp-ext -> dist
generate: n_ctx = 2048, n_batch = 1024, n_predict = 512, n_keep = 0

== Running in interactive mode. ==
- Press Ctrl+C to interject at any time.
- Press Return to return control to the AI.
- To return control without starting a new line, end your input with '/'.
- If you want to submit another line, end your input with '\n'.

system
你是编程小助手
> hello
Hello! How can I assist you today?
>
```

代码调用

先启动服务端

```
llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -c 2048 --threads 8 \
--port 8081
```

对话程序:

```
cd ~/dev_ws/src/yzbot/tools_demo/tools_demo
python3 chat_interactive.py
```

```

ros@u22:~/dev_ws/src/yzbot/tools_demo/tools_demo$ python3 chat_interactive.py
=== llama 交互式聊天程序 ===
输入 'quit' 或 'exit' 退出程序
-----

> hello
正在思考...

助手: Hello! How can I assist you today?
>

```

接入 ros

```
ros2 run tools_demo chat_ros
```

```

ros@u22:~/dev_ws/src/yzbot/tools_demo/tools_demo$ python3 chat_interactive.py
=== llama 交互式聊天程序 ===
输入 'quit' 或 'exit' 退出程序
-----

> hello
正在思考...

助手: Hello! How can I assist you today?
> ^C
程序被中断,再见!
ros@u22:~/dev_ws/src/yzbot/tools_demo/tools_demo$ ros2 run tools_demo chat_ros
=== llama 交互式聊天程序 ===
输入 'quit' 或 'exit' 退出程序
每次回复后可选择处理方式:
  N - 不处理
  R - 发布到话题
-----

> I need blue
正在思考...

助手: {"color": "blue","num": 5}

选择处理方式 [N/R]: R
发布到话题...
已发布消息: {"color": "blue","num": 5}
> []

```

聊天返回的结果会发布到/llama_command,你可以自定义提示词

~/dev_ws/src/yzbot/tools_demo/tools_demo/chat_interactive_ros.py

```

1 import requests
2 from std_msgs.msg import String
3 import rclpy
4 from rclpy.node import Node
5 import json
6
7 class LlamaChat(Node):
8     """LlamaChat 类,
9     需要启动 llama-server -m /home/sunrise/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -c 2048 --threads 8 --port 8081
10
11
12     def __init__(self, base_url="http://localhost:8081"):
13         super().__init__("LlamaChat")
14         self.base_url = base_url
15         self.chat_history = []
16         self.system_prompt = '你是编程小助手，我会告诉你识别的物体颜色和编号，你按照{"color": "red", "num": 5} 这样的格式回复；比如蓝色4号：{"color": "blue", "num": 4}。'
17         # 你的回复必须严格遵循这个格式。'
18
19     # 创建话题发布者（替换服务客户端）
20     self.command_publisher = self.create_publisher(String, "llama_command", 10)
21
22     print("=== Llama 交互式聊天程序 ===")
23     print("输入 'quit' 或 'exit' 退出程序")
24     print("每次回复后可选择处理方式:")
25     print("N - 不处理")
26     print("R - 发布到话题")
27     print("-" * 40)
28     while True:
29         try:
30             # 获取用户输入
31             user_input = input("\n> ").strip()

```

测试机械臂运动 (Action):

- 机械臂运动指令示例:

```

Plain Text
ros2 action send_goal /arm_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{...}"

```

- 夹爪运动指令示例:

```

Plain Text
ros2 action send_goal
/gripper_controller/follow_joint_trajectory
control_msgs/action/FollowJointTrajectory "{...}"

```

- 使用测试脚本 `send_goal_pick.sh`，通过发送 Action Goal 来控制机械臂和夹爪。

```

cd ~/dev_ws/src/yzbot
./pick_off.sh #控制夹爪关节夹取
./pick_on.sh #控制夹爪关节张开
./my_send_goal.sh # 控制机械臂关节移动

```

夹爪夹取

- 吸附指令:

```
ros2 service call /ATTACHLINK linkattacher_msgs/srv/AttachLink "{model1_name: 'six_arm', link1_name: 'link6', model2_name: 'red_cube_1', link2_name: 'link'}"
```

- 分离指令:

```
ros2 service call /DETACHLINK linkattacher_msgs/srv/DetachLink "{model1_name: 'six_arm', link1_name: 'link6', model2_name: 'red_cube_1', link2_name: 'link'}"
```

- 可以使用脚本实现对指定方块的夹取和释放:

```
cd /home/ros/dev_ws/src/yzbot
ros@u22:~/dev_ws/src/yzbot$ ./cube_control.sh
用法: ./cube_control.sh [on|off] [red|blue] [1-5]
ros@u22:~/dev_ws/src/yzbot$ ./cube_control.sh on red 5
```

吸附 红色 id5 的方块